



NextGen

Efficient Python Lecture 05

CERN
STEAM Academy

10 July 2026

Data Pipelines and Labelled Analysis

Jacek Gatlik

email: jacek.gatlik@agh.edu.pl

www: <https://galaxy.agh.edu.pl/~jgatlik/>

Faculty of Physics and Applied Computer Science,
AGH University of Krakow, Poland



Outline

Labelled tables and data semantics with pandas

Vectorized table transformations

Small reproducible analysis pipelines

Advanced pandas patterns

Polars for columnar and lazy data pipelines

Further reading

Section 1

Labelled tables and data semantics with pandas

Why pandas, if we already know NumPy?

NumPy is excellent when the numerical structure is naturally array-like: grids, fields, matrices, tensors, and homogeneous numerical data.

Many scientific datasets, however, are tabular:

- ▶ one row per measurement, event, simulation run, or observation,
- ▶ columns with different meanings and possibly different types,
- ▶ labels such as detector name, sample type, run identifier, or timestamp,
- ▶ missing values, categories, and metadata-like columns.

pandas adds a labelled, tabular layer on top of array-based computation. It is not a replacement for NumPy; it is a tool for the stage where data have names and structure.

A table is more than a two-dimensional array

A two-dimensional NumPy array can store numbers, but it does not remember what the columns mean.

```
1 import numpy as np
2
3 A = np.array([
4     [300.1, 0.15, 12.4],
5     [301.0, 0.20, 13.1],
6     [299.7, 0.10, 11.8],
7 ])
```

Which column is temperature? Which one is magnetic field? Which one is signal?

A labelled table keeps this information close to the data. This is especially important before plotting, fitting, exporting, or preparing data for machine learning.

Creating a small DataFrame

A DataFrame is a table with labelled columns and labelled rows.

```
1 import pandas as pd
2
3 df = pd.DataFrame({
4     "run_id": ["run_001", "run_002", "run_003"],
5     "temperature_K": [300.1, 301.0, 299.7],
6     "field_T": [0.15, 0.20, 0.10],
7     "signal": [12.4, 13.1, 11.8],
8 })
9
10 df
```

The column names are part of the analysis. They document the meaning of the values and make later operations less dependent on remembering column positions.

Series: one labelled quantity

A Series is a one-dimensional labelled array. A column of a DataFrame is usually a Series.

```
1 signal = df["signal"]
2
3 signal
4 signal.index
5 signal.values
```

The values can be used by NumPy, but the index and name remain available.

```
1 mean_signal = signal.mean()
2 centered = signal - mean_signal
```

This is still vectorized computation, but now the vector has a label and an index.

DataFrame **axes:** rows and columns

A DataFrame has two labelled axes:

- ▶ the row index,
- ▶ the column index.

```
1 df.index  
2 df.columns  
3 df.shape
```

This looks similar to NumPy, but the interpretation is different. In a scientific table, columns usually represent named quantities, while rows represent observations, measurements, events, or simulation runs.

The labels make many operations safer and more readable than using only integer positions.

Index labels are not row numbers

The row index can contain labels. These labels do not have to be consecutive integers.

```
1 df2 = df.set_index("run_id")
2
3 df2.loc["run_002"]    # label-based access
4 df2.iloc[1]           # position-based access
```

The distinction between `loc` and `iloc` is important:

- ▶ `loc` uses labels,
- ▶ `iloc` uses integer positions.

Confusing labels with positions is one of the common sources of subtle errors in table analysis.

Column selection returns labelled objects

Selecting one column returns a `Series`. Selecting several columns returns a new `DataFrame`.

```
1 signal = df["signal"]
2
3 features = df[["temperature_K", "field_T"]]
```

This distinction matters because a single column and a table with one column have different shapes and different behaviour.

```
1 df["signal"].shape      # (3,)
2 df[["signal"]].shape    # (3, 1)
```

This is closely analogous to shape awareness in NumPy. The object type carries information about the intended structure.

Data types carry assumptions

Each column has a data type. The dtype determines how values are stored and which operations make sense.

```
1 df.dtypes
```

Typical column types include:

- ▶ floating-point numbers for measured quantities,
- ▶ integers for counts,
- ▶ booleans for masks or flags,
- ▶ strings for identifiers,
- ▶ categories for repeated labels,
- ▶ datetimes for time stamps.

A wrong dtype often means that the data were not read as intended. Inspecting dtypes is one of the first checks after loading a table.

Missing values are data, not only errors

Missing values may represent failed measurements, unavailable metadata, detector cuts, or intentionally undefined quantities. They should be handled explicitly.

```
1 df["signal"].isna()  
2  
3 n_missing = df["signal"].isna().sum()
```

Possible responses depend on the scientific meaning:

- ▶ remove rows where a crucial quantity is missing,
- ▶ fill missing values using a justified rule,
- ▶ keep missingness as information,
- ▶ investigate whether missingness is systematic.

A missing value is not automatically a nuisance. Sometimes it is a diagnostic.

Categorical data

Many table columns contain repeated labels: detector names, sample types, phases, run modes, boundary-condition choices, or simulation families.

```
1 df["detector"] = ["A", "A", "B"]
2
3 df["detector"] = df["detector"].astype("category")
4
5 df["detector"].cat.categories
```

Categorical columns make the intended meaning clearer: the column is not arbitrary text, but a finite set of labels.

This becomes important for grouping, plotting, joins, statistical summaries, and preparing features for machine learning.

Reading a CSV as a typed table

Reading data is not only opening a file. It is the point where raw text becomes a structured table.

```
1 from pathlib import Path
2 import pandas as pd
3
4 path = Path("data/raw/measurements.csv")
5
6 dtypes = {
7     "run_id": "string",
8     "detector": "category",
9     "count": "Int64",
10 }
11
12 df = pd.read_csv(
13     path,
14     dtype=dtypes,
15     parse_dates=["time"],
16 )
```

Whenever possible, important types should be specified deliberately rather than discovered accidentally.

Inspect before analysis

After loading a table, do not immediately start fitting, plotting, or training a model. First inspect the structure.

```
1 df.head()
2 df.info()
3
4 df.describe()
5
6 df.isna().sum()
7 df.dtypes
```

Useful first questions:

- ▶ Are the expected columns present?
- ▶ Do the dtypes make sense?
- ▶ Are there missing or non-finite values?
- ▶ Are the numerical ranges physically plausible?

This is the tabular analogue of inspecting array shape, dtype, and finite values before a numerical computation.

From raw columns to meaningful columns

A good table should make scientific meaning visible. This often requires renaming columns, converting units, and creating derived quantities.

```
1 df = df.rename(columns={
2     "T": "temperature_K",
3     "B": "field_T",
4     "S": "signal",
5 })
6
7 df["temperature_C"] = df["temperature_K"] - 273.15
8
9 df["signal_rate"] = df["signal"] / df["exposure_s"]
```

The raw data are not destroyed. We add columns that express the quantities needed by the analysis.

This is the point where `pandas` becomes more than a file-reading tool: it becomes a labelled, vectorized layer for scientific reasoning.

Section 2

Vectorized table transformations

From labelled data to transformations

Once the table has meaningful columns, the next step is not to loop over rows. The next step is to express transformations in terms of columns, masks, groups, and joins.

In a scientific analysis, we often want to:

- ▶ select physically meaningful measurements,
- ▶ remove or mark invalid values,
- ▶ compute derived quantities,
- ▶ aggregate repeated measurements,
- ▶ combine event-level data with metadata,
- ▶ reshape data for plotting, modelling, or export.

The goal is to keep the analysis close to the structure of the data, rather than writing ad hoc row-wise scripts.

Filtering with Boolean masks

A Boolean mask is a labelled array of `True/False` values. It selects rows that satisfy a condition.

```
1 mask = (  
2     (df["energy_GeV"] > 5.0)  
3     & (df["chi2"] < 2.5)  
4     & df["signal"].notna()  
5 )  
6  
7 selected = df.loc[mask].copy()
```

The parentheses are important. In pandas, conditions are combined with `&`, `|`, and `~`, not with ordinary Python `and`, `or`, and `not`.

The result is still a `DataFrame`: a labelled table, not just a list of rows.

Readable filters with `query`

For some filters, `query` can make the condition easier to read.

```
1 min_energy = 5.0
2 max_chi2 = 2.5
3
4 selected = df.query(
5     "energy_GeV > @min_energy and chi2 < @max_chi2"
6 )
```

The symbol `@` means that the value is taken from the surrounding Python scope.

This style is useful when the condition is mostly a readable expression involving columns. For more complex logic, explicit Boolean masks are usually clearer and easier to debug.

Avoid row-wise iteration for column operations

In tabular analysis, a common beginner mistake is to treat every row as a separate Python object.

```
1 # Avoid this style for ordinary column operations:  
2 # for _, row in df.iterrows():  
3 #     compute something from row["energy_GeV"]
```

Most operations should be expressed directly on columns.

```
1 p2 = df["energy_GeV"]**2 - df["mass_GeV"]**2  
2  
3 df["momentum_GeV"] = np.sqrt(p2.clip(lower=0.0))
```

This is faster, shorter, and usually closer to the mathematical structure of the operation.

Controlled assignment

When modifying a table, make it clear whether we are updating the original object or creating a new one.

```
1 df = df.copy()
2
3 df["accepted"] = False
4 df.loc[mask, "accepted"] = True
```

If we create a selected table and then modify it, it is safer to make an explicit copy.

```
1 accepted = df.loc[df["accepted"]].copy()
2
3 accepted["weight"] = 1.0 / accepted["sigma"]**2
```

The point is not only to satisfy pandas. The point is to make the data flow visible: which table is being changed, and which table is a derived product?

Creating derived columns with `assign`

The method `assign` is useful when derived columns are part of a readable transformation chain.

```
1 clean = (  
2     df.rename(columns={"E": "energy_GeV"})  
3     .assign(  
4         signal_rate=lambda d: d["signal"] / d["exposure_s"],  
5         accepted=lambda d: (  
6             (d["energy_GeV"] > 5.0)  
7             & (d["chi2"] < 2.5)  
8         ),  
9     )  
10 )
```

The lambda functions receive the current table in the chain. This makes it possible to define new columns using columns that already exist in the transformed table.

Cleaning physical ranges

Cleaning is not only a technical operation. It encodes assumptions about what values are physically meaningful.

```
1 numeric_cols = ["energy_GeV", "chi2", "signal"]
2
3 finite = np.isfinite(df[numeric_cols]).all(axis=1)
4
5 physical = (
6     (df["energy_GeV"] >= 0.0)
7     & (df["exposure_s"] > 0.0)
8     & (df["chi2"] >= 0.0)
9 )
10
11 clean = df.loc[finite & physical].copy()
```

A good analysis should not silently remove data without justification. Invalid rows should be counted, inspected, or saved as diagnostics.

→ **Example 05 - 01**

Sorting and selecting representative rows

Sorting is often used for inspection, selecting extreme cases, or preparing time-ordered data.

```
1 largest = clean.nlargest(5, "signal_rate")
2
3 smallest_chi2 = clean.nsmallest(5, "chi2")
4
5 time_ordered = clean.sort_values(["run_id", "time_s"])
6
7 first_event = time_ordered.groupby("run_id").head(1)
```

These operations are useful during exploratory analysis, but they are also useful diagnostics. Extreme rows often reveal unit errors, failed measurements, duplicated entries, or unexpected parameter settings.

Aggregation as labelled reduction

Aggregation reduces many rows to fewer rows. Conceptually, it is similar to a NumPy reduction, but the reduction is performed over labelled groups.

```
1 mean_signal = clean["signal"].mean()
2
3 mean_by_detector = (
4     clean.groupby("detector")["signal"]
5     .mean()
6 )
```

The first expression reduces one column to one number. The second expression reduces one column separately for each detector.

This is one of the central ideas of pandas: reductions can respect labels and categories.

groupby: **split, apply, combine**

The standard mental model for groupby is:

split by labels $\longrightarrow$ apply a reduction $\longrightarrow$ combine the results

```
1 run_summary = (  
2     clean.groupby(["run_id", "detector"],  
3                   as_index=False,  
4                   observed=True)  
5     .agg(  
6         n_events=("event_id", "size"),  
7         mean_energy_GeV=("energy_GeV", "mean"),  
8         std_signal=("signal", "std"),  
9         accepted_fraction=("accepted", "mean"),  
10    )  
11 )
```

The result is again a table, now with one row per group.

Binning continuous variables

Continuous quantities are often grouped into physically meaningful ranges: energy intervals, temperature ranges, spatial shells, or time windows.

```
1 bins = [0.0, 2.0, 5.0, 10.0, np.inf]
2 labels = ["low", "medium", "high", "very high"]
3
4 clean = clean.assign(
5     energy_bin=pd.cut(
6         clean["energy_GeV"],
7         bins=bins,
8         labels=labels,
9     ))
10
11 binned = (
12     clean.groupby("energy_bin", observed=True)
13     .agg(
14         n=("signal", "size"),
15         mean_signal=("signal", "mean"),
16     ))
```

Binning is a modelling choice. The bin edges should be chosen deliberately, not only because they produce a nice plot.

Joining data with metadata

Scientific tables often come in layers. One table may contain event-level measurements, while another contains run-level metadata.

```
1 events = pd.read_csv(  
2     "data/raw/events.csv",  
3     dtype={"run_id": "string"},  
4 )  
5  
6 runs = pd.read_csv(  
7     "data/raw/runs.csv",  
8     dtype={"run_id": "string"},  
9 )  
10  
11 events = events.merge(  
12     runs[["run_id", "field_T", "temperature_K"]],  
13     on="run_id",  
14     how="left",  
15     validate="many_to_one",  
16 )
```

The event table now contains both measured quantities and the experimental conditions associated with each run.

Validate joins

A join can silently duplicate rows or leave missing metadata if the key structure is not what we think it is.

```
1 merged = events.merge(  
2     runs,  
3     on="run_id",  
4     how="left",  
5     validate="many_to_one",  
6     indicator=True,  
7 )  
8  
9 merged["_merge"].value_counts()  
10  
11 missing_meta = merged.loc[  
12     merged["_merge"] != "both",  
13     "run_id",  
14 ].unique()
```

The argument `validate` expresses our expectation about the relation between the tables. This is a simple but powerful protection against accidental many-to-many joins.

→ **Example 05 - 02**

Long and wide data

The same data can be represented in different table shapes.

A wide table has one column per measured channel.

```
1 wide = pd.DataFrame({  
2     "time_s": time,  
3     "det_A": signal_A,  
4     "det_B": signal_B,  
5 })
```

A long table has one row per observation and uses a label column.

```
1 long = wide.melt(  
2     id_vars="time_s",  
3     var_name="detector",  
4     value_name="signal",  
5 )
```

Long tables are often better for grouping, plotting, joining, and machine-learning preparation.

Pivot tables as labelled grids

A pivot operation can transform a long table into a labelled grid.

```
1 grid = long.pivot_table(  
2     index="time_s",  
3     columns="detector",  
4     values="signal",  
5     aggfunc="mean",  
6 )  
7  
8 U = grid.to_numpy()  
9 times = grid.index.to_numpy()  
10 detectors = grid.columns
```

This is the moment where pandas and NumPy meet naturally. The table stores labels and semantics; the array `U` can be used for numerical operations, plotting, or model input.

The important habit is to keep track of what the axes mean after leaving pandas.

Section 3

Small reproducible analysis pipelines

From labelled tables to NumPy arrays

Many numerical and machine-learning tools expect NumPy arrays. A good workflow converts a labelled table to arrays deliberately.

```
1 feature_cols = [  
2     "energy_GeV",  
3     "chi2",  
4     "temperature_K",  
5 ]  
6  
7 target_col = "accepted"  
8  
9 X = clean.loc[:, feature_cols].to_numpy(dtype=float)  
10 y = clean.loc[:, target_col].to_numpy(dtype=bool)
```

The list `feature_cols` is part of the analysis. It records which columns became features and in which order.

Preprocessing without losing the labels

Preprocessing can be computed in pandas and then exported to NumPy. This keeps the meaning of each feature visible.

```
1 train_mask = clean["split"] == "train"
2
3 mu = clean.loc[train_mask, feature_cols].mean()
4 scale = clean.loc[train_mask, feature_cols].std(ddof=0)
5 scale = scale.replace(0.0, 1.0)
6
7 X_scaled_table = (clean[feature_cols] - mu) / scale
8
9 X_scaled = X_scaled_table.to_numpy(dtype=float)
```

The statistics are fitted on the training subset only. The labels help us avoid mixing up feature order or scaling the wrong column.

Saving preprocessing parameters

If preprocessing is part of the model input, its parameters should be saved. Otherwise, the model cannot be applied consistently later.

```
1 scaling_table = pd.DataFrame({
2     "feature": feature_cols,
3     "mean": mu.loc[feature_cols].to_numpy(),
4     "scale": scale.loc[feature_cols].to_numpy(),
5 })
6
7 scaling_table.to_csv(
8     "results/tables/feature_scaling.csv",
9     index=False,
10 )
```

This is a small example of a larger principle: an analysis output is not only a figure or a final number. It is also the information needed to reproduce how that result was obtained.

→ **Example 05 - 03**

Plotting labelled summaries

A common workflow is to aggregate a table and then plot the summary.

```
1 summary = (  
2     clean.groupby("field_T", as_index=False)  
3     .agg(  
4         mean_signal=("signal", "mean"),  
5         std_signal=("signal", "std"),)  
6  
7 fig, ax = plt.subplots()  
8  
9 ax.errorbar(  
10     summary["field_T"],  
11     summary["mean_signal"],  
12     yerr=summary["std_signal"],  
13     fmt="o",)  
14  
15 ax.set(  
16     xlabel="magnetic field [T]",  
17     ylabel="mean signal",)
```

The plot is built from named columns. This reduces the risk of accidentally plotting the wrong quantity on the wrong axis.

An end-to-end pipeline

The whole lecture can be summarized as a small pipeline:

raw events + run metadata $\longrightarrow$ clean table $\longrightarrow$ summary table $\longrightarrow$ figure and saved outputs

A useful implementation separates these stages into small functions:

- ▶ load the inputs,
- ▶ clean and enrich the event table,
- ▶ compute summaries,
- ▶ save tables and configuration,
- ▶ make diagnostic plots.

This structure is simple, but it is already much more reproducible than a long notebook with hidden state.

Pipeline step: load inputs

The loading function defines where raw data enter the workflow.

```
1 from pathlib import Path
2 import pandas as pd
3
4 def load_inputs(project_root):
5     raw_dir = project_root / "data" / "raw"
6
7     events = pd.read_csv(
8         raw_dir / "events.csv",
9         dtype={
10             "run_id": "string",
11             "detector": "category",},,)
12
13     runs = pd.read_csv(
14         raw_dir / "runs.csv",
15         dtype={"run_id": "string"},)
16
17     return events, runs
```

The function does not clean the data. It only loads the inputs and makes the first type assumptions explicit.

Pipeline step: clean and enrich

The cleaning function encodes the selection rules and joins the event table with metadata.

```
1 import numpy as np
2
3 def clean_events(events, runs, config):
4     df = events.merge(
5         runs,
6         on="run_id",
7         how="left",
8         validate="many_to_one",)
9     numeric = ["energy_GeV", "chi2", "signal", "exposure_s"]
10    mask = np.isfinite(df[numeric]).all(axis=1)
11    mask &= df["energy_GeV"] >= config["min_energy_GeV"]
12    mask &= df["chi2"] <= config["max_chi2"]
13    mask &= df["exposure_s"] > 0.0
14    clean = df.loc[mask].assign(
15        signal_rate=lambda d: d["signal"] / d["exposure_s"])
16    return clean.copy()
```

The scientific cuts are now parameters, not hidden constants scattered through the notebook.

Pipeline step: summarize

The summary function turns event-level data into a smaller table of diagnostics.

```
1 def summarize(clean):
2     summary = (
3         clean.groupby(["run_id", "detector"],
4                        as_index=False,
5                        observed=True)
6         .agg(
7             n_events=("signal", "size"),
8             mean_signal=("signal", "mean"),
9             std_signal=("signal", "std"),
10            mean_rate=("signal_rate", "mean"),
11            field_T=("field_T", "first"),
12        )
13     )
14
15     return summary
```

The output is a regular DataFrame. It can be inspected, saved, plotted, joined with other summaries, or passed to later analysis.

Pipeline step: save tables and configuration

Saving outputs should be explicit. The result table and the configuration belong together.

```
1 import json
2
3 def save_outputs(summary, config, project_root):
4     table_dir = project_root / "results" / "tables"
5     table_dir.mkdir(parents=True, exist_ok=True)
6
7     summary.to_csv(
8         table_dir / "run_summary.csv",
9         index=False,
10    )
11
12    config_file = table_dir / "config.json"
13    config_file.write_text(
14        json.dumps(config, indent=2),
15        encoding="utf8",
16    )
```

This makes it possible to interpret the output file without guessing which cuts or thresholds produced it.

Pipeline step: make a diagnostic plot

A plot function should receive the data it needs and save the figure in a controlled location.

```
1 import matplotlib.pyplot as plt
2
3 def plot_summary(summary, project_root):
4     fig_dir = project_root / "results" / "figures"
5     fig_dir.mkdir(parents=True, exist_ok=True)
6     by_field = (
7         summary.groupby("field_T", as_index=False)
8         .agg(mean_rate=("mean_rate", "mean")))
9     fig, ax = plt.subplots()
10    ax.plot(by_field["field_T"], by_field["mean_rate"], "o-")
11    ax.set(xlabel="magnetic field [T]", ylabel="mean signal rate",)
12
13    fig.savefig(fig_dir / "signal_rate_by_field.pdf", bbox_inches="tight")
14    return fig, ax
```

The function returns the figure and axis, so the caller can still inspect or modify the plot if needed.

→ **Example 05 - 04**

Pipeline runner

The runner function connects the stages. It should read like the outline of the analysis.

```
1 def run_pipeline(project_root, config):
2     events, runs = load_inputs(project_root)
3
4     clean = clean_events(events, runs, config)
5
6     summary = summarize(clean)
7
8     save_outputs(summary, config, project_root)
9
10    fig, ax = plot_summary(summary, project_root)
11
12    return clean, summary, fig, ax
```

This is not a large software framework. It is just enough structure to make the analysis repeatable, inspectable, and easier to modify.

Practical checklist for tabular pipelines

Before trusting a pandas-based analysis, it is useful to ask:

- ▶ Are raw, processed, and result files separated?
- ▶ Are paths constructed portably from a project root?
- ▶ Are dtypes inspected after reading the data?
- ▶ Are missing and non-finite values handled deliberately?
- ▶ Are physical cuts visible as parameters?
- ▶ Are joins validated?
- ▶ Are groupby operations producing the intended groups?
- ▶ Are feature columns recorded before conversion to NumPy?
- ▶ Are output tables, figures, and configuration saved together?

The main message is simple: pandas is not only a convenient table library. Used carefully, it is a labelled layer for reproducible scientific data analysis.

Section 4

Advanced pandas patterns

What pandas should leave behind

At this point, `pandas` should not be viewed only as a convenient CSV reader. In a scientific workflow, it is a labelled transformation layer.

raw table → typed table → clean table → diagnostics → arrays, figures, or summaries.

The most important habits are:

- ▶ keep column names scientifically meaningful,
- ▶ inspect dtypes and missing values before analysis,
- ▶ express transformations column-wise, not row-wise,
- ▶ make grouping, joining, and reshaping choices explicit,
- ▶ save the intermediate information needed to reproduce the result.

The advanced part is not a new syntax trick. It is making the table itself a reliable object in the analysis.

Group-wise diagnostics with transform

Aggregation reduces each group to one row. Sometimes we need group-level information broadcast back to every original row. This is the role of `transform`.

```
1 g = clean.groupby("run_id")["signal"]
2
3 clean = clean.assign(
4     signal_run_mean=g.transform("mean"),
5     signal_run_std=g.transform("std"),
6 )
7
8 clean["signal_z"] = (
9     (clean["signal"] - clean["signal_run_mean"])
10    / clean["signal_run_std"].replace(0.0, 1.0)
11 )
```

Now each event knows how its signal compares with other events from the same run. This is useful for normalization, outlier detection, run-by-run calibration, detector corrections, and diagnostics that should respect the experimental grouping structure.

Readable pipelines with pipe

When a chain becomes long, `pipe` lets us name the stages of the analysis without hiding the data flow.

```
1 def add_signal_rate(df):
2     return df.assign(
3         signal_rate=lambda d: d["signal"] / d["exposure_s"]
4     )
5 def apply_quality_cuts(df, config):
6     mask = (
7         (df["energy_GeV"] >= config["min_energy_GeV"])
8         & (df["chi2"] <= config["max_chi2"])
9         & df["signal"].notna()
10    )
11    return df.loc[mask].copy()
12 clean = (
13     raw
14     .pipe(add_signal_rate)
15     .pipe(apply_quality_cuts, config=config)
16 )
```

The pipeline remains ordinary Python. The advantage is that each stage has a name, can be tested separately, and can be reused in a script or notebook.

Time-aware analysis

Many scientific tables contain time stamps: detector events, simulation snapshots, logged measurements, or repeated calibrations. Sorting and grouping should be deliberate.

```
1 events = events.sort_values(["run_id", "time"])
2
3 events["rolling_signal"] = (
4     events.groupby("run_id")["signal"]
5         .rolling(window=5, min_periods=1)
6         .mean()
7         .reset_index(level=0, drop=True)
8 )
```

This computes a rolling average separately inside each run. The grouping matters: without it, the end of one run could be averaged with the beginning of another.

Time-aware table analysis is still numerical analysis. Ordering, window size, and grouping are part of the method.

When pandas stops being the whole answer

`pandas` is excellent for many small and medium-sized scientific tables, especially when exploration, flexibility, and ecosystem compatibility matter.

But some workflows expose limits:

- ▶ files are large compared with memory,
- ▶ only a few columns are needed from a wide table,
- ▶ many filters and projections happen before aggregation,
- ▶ Python-level callbacks or row-wise operations become the bottleneck,
- ▶ the analysis should behave more like a query plan than a sequence of immediate steps.

This does not mean that `pandas` is wrong. It means that the workload may benefit from a different execution model.

This is where `Polars` becomes useful.

Section 5

Polars for columnar and lazy data pipelines

Why discuss Polars?

Polars is another DataFrame library, but its design is different from pandas. It is built around typed columns, expression-based transformations, lazy query plans, and parallel execution.

table API + query optimizer + columnar execution $\longrightarrow$ fast labelled analysis.

For scientific work, the most relevant question is not whether Polars is always faster. The relevant question is:

Can this data workflow be expressed as a typed, column-wise, optimizable pipeline?

If the answer is yes, Polars can be a very good fit.

First contact: a typed DataFrame

A Polars table looks familiar, but the emphasis on schema is stronger.

```
1 import polars as pl
2
3 df = pl.DataFrame({
4     "run_id": ["run_001", "run_002", "run_003"],
5     "temperature_K": [300.1, 301.0, 299.7],
6     "field_T": [0.15, 0.20, 0.10],
7     "signal": [12.4, 13.1, 11.8],
8 })
9
10 print(df.schema)
```

The schema tells us the column names and data types. This is the tabular analogue of checking shape and dtype for NumPy arrays.

For scientific data, the schema is part of the contract: it says what kind of quantities the pipeline expects.

Columnar thinking

Polars is designed for column-oriented operations. This matches many scientific table workflows, where we compute with whole columns rather than with individual rows.

```
1 df2 = df.with_columns(  
2     temperature_C=pl.col("temperature_K") - 273.15,  
3     scaled_signal=pl.col("signal") / pl.col("signal").mean(),  
4 )  
5  
6 summary = df2.select(  
7     pl.col("signal").mean().alias("mean_signal"),  
8     pl.col("signal").std().alias("std_signal"),  
9 )
```

The expressions `pl.col(...)` describe computations on columns. They are not Python loops over rows.

This expression style is one of the main differences between writing table transformations as procedural code and writing them as a query.

Eager and lazy execution

Polars has two important execution modes.

- ▶ Eager mode: each operation is executed immediately.
- ▶ Lazy mode: operations are collected into a query plan and executed later.

```
1 # Eager: execute now  
2 df_eager = pl.read_csv("data/raw/events.csv")  
3  
4 # Lazy: build a query plan  
5 q = (  
6     pl.scan_csv("data/raw/events.csv")  
7     .filter(pl.col("energy_GeV") > 5.0)  
8     .select(["run_id", "detector", "energy_GeV", "signal"])  
9 )  
10  
11 # Execute the plan  
12 result = q.collect()
```

Lazy execution is important because the library can inspect the whole plan before doing the work.

Lazy queries are analysis plans

A lazy query is not the result. It is a description of how to obtain the result.

```
1 q = (  
2     pl.scan_parquet("data/processed/events.parquet")  
3     .filter(pl.col("chi2") < 2.5)  
4     .with_columns(  
5         signal_rate=pl.col("signal") / pl.col("exposure_s")  
6     )  
7     .group_by("run_id")  
8     .agg(  
9         n_events=pl.len(),  
10        mean_rate=pl.col("signal_rate").mean(),  
11    )  
12 )  
13  
14 print(q.explain())
```

The method `explain()` shows the query plan. This is useful pedagogically and practically: it helps us see what the engine intends to do.

In a scientific workflow, the query plan is a computational object, not only an implementation detail.

Projection and predicate pushdown

One reason lazy execution can be faster is that unnecessary work can be removed before the file is fully read.

```
1 q = (  
2     pl.scan_parquet("data/raw/events.parquet")  
3     .filter(pl.col("energy_GeV") > 5.0)  
4     .select([  
5         "run_id",  
6         "detector",  
7         "energy_GeV",  
8         "signal",  
9     ])  
10 )
```

Two optimizations are especially important:

- ▶ predicate pushdown: apply filters as early as possible,
- ▶ projection pushdown: read only the columns that are actually needed.

For large files, this can reduce both memory use and I/O. The scientific meaning is unchanged; the engine simply avoids reading data that cannot affect the result.

Typed reading instead of accidental parsing

Just as in pandas, reading a file is the point where raw text becomes typed data. In Polars, we can specify important schema information explicitly.

```
1 schema_overrides = {  
2     "run_id": pl.String,  
3     "detector": pl.Categorical,  
4     "count": pl.Int64,  
5     "signal": pl.Float64,  
6 }  
7  
8 events = pl.scan_csv(  
9     "data/raw/events.csv",  
10    schema_overrides=schema_overrides,  
11    try_parse_dates=True,  
12 )
```

This is especially useful when a column should not be guessed from the first rows of a file. For scientific data, accidental types can be dangerous: a run identifier is not a number just because it looks like one, and a categorical label is not arbitrary free text.

Filtering and derived columns

A typical Polars cleaning step combines filters and derived columns as expressions.

```
1 clean = (  
2     pl.scan_parquet("data/raw/events.parquet")  
3     .filter(  
4         (pl.col("energy_GeV") >= 5.0)  
5         & (pl.col("chi2") <= 2.5)  
6         & pl.col("signal").is_not_null()  
7         & pl.col("exposure_s").is_not_null()  
8     )  
9     .with_columns(  
10        signal_rate=pl.col("signal") / pl.col("exposure_s"),  
11        accepted=pl.lit(True),  
12    )  
13 )
```

The syntax is close to the pandas mask style, but the expressions are part of a query plan. They can be optimized before execution.

Grouped summaries

Group-by aggregation in Polars follows the same scientific logic as in pandas: split the table by labels and reduce each group to diagnostics.

```
1 run_summary = (  
2     clean  
3     .group_by(["run_id", "detector"])  
4     .agg(  
5         n_events=pl.len(),  
6         mean_energy_GeV=pl.col("energy_GeV").mean(),  
7         std_signal=pl.col("signal").std(),  
8         mean_rate=pl.col("signal_rate").mean(),  
9         first_temperature_K=pl.col("temperature_K").first(),  
10    )  
11 )
```

The result is still lazy if the input is lazy. No computation has to happen until we call `collect()`.

This means that filtering, column selection, derived quantities, and grouping can be optimized as one pipeline.

Joins as part of the query

Scientific data often combine event-level measurements with run-level metadata. In Polars, joins can also be part of the lazy plan.

```
1 events = pl.scan_parquet("data/raw/events.parquet")
2 runs = pl.scan_parquet("data/raw/runs.parquet")
3
4 q = (
5     events
6     .join(
7         runs.select(["run_id", "field_T", "temperature_K"]),
8         on="run_id",
9         how="left",
10    )
11    .filter(pl.col("temperature_K").is_not_null())
12 )
```

The same scientific warning remains: a join encodes an assumption about the relation between tables.

After a join, always check whether rows were duplicated unexpectedly or whether metadata are missing.

Missing values and invalid values

Polars distinguishes null values from floating-point NaN. In scientific data, both may matter, but they usually mean different things.

```
1 diagnostics = clean.select(  
2     pl.all().null_count(),  
3 )  
4  
5 finite_clean = clean.filter(  
6     pl.col("signal").is_finite()  
7     & pl.col("energy_GeV").is_finite()  
8     & (pl.col("energy_GeV") >= 0.0)  
9 )
```

A null value often means missing data. A NaN often means invalid numerical output. Both should be treated as diagnostics, not silently ignored.

Streaming and larger-than-memory workflows

Lazy execution can also enable streaming execution for compatible queries. Instead of materializing the full dataset at once, the engine can process data in batches.

```
1 q = (  
2     pl.scan_parquet("data/raw/events.parquet")  
3     .filter(pl.col("energy_GeV") > 5.0)  
4     .group_by("detector")  
5     .agg(  
6         n_events=pl.len(),  
7         mean_signal=pl.col("signal").mean(),  
8     )  
9 )  
10  
11 summary = q.collect(engine="streaming")
```

This is useful when the full file is too large to fit comfortably in memory, but the final result is a much smaller summary.

Streaming is not magic. The query must be compatible with streaming execution, and the workflow should still be validated on a smaller test case.

Parallelism without manual parallel code

In many Polars workflows, the user writes expressions and the engine decides how to execute parts of the plan efficiently.

```
1 q = (  
2     pl.scan_parquet("data/raw/events.parquet")  
3     .with_columns(  
4         p2=pl.col("energy_GeV")**2 - pl.col("mass_GeV")**2  
5     )  
6     .with_columns(  
7         momentum_GeV=pl.when(pl.col("p2") > 0.0)  
8             .then(pl.col("p2").sqrt())  
9             .otherwise(0.0)  
10    )  
11    .group_by("run_id")  
12    .agg(pl.col("momentum_GeV").mean())  
13 )
```

The important point is that we are not writing threads or processes manually. We are giving the engine a columnar query that exposes opportunities for parallel execution. This is often safer than adding manual parallelism too early.

Interoperability with pandas and NumPy

A Polars pipeline does not have to replace the whole ecosystem. It can be used for the part of the workflow where it helps most.

```
1 summary_pl = q.collect()
2
3 # Convert to pandas for a library that expects pandas
4 summary_pd = summary_pl.to_pandas()
5
6 # Convert selected columns to NumPy for numerical modelling
7 feature_cols = ["energy_GeV", "chi2", "signal_rate"]
8
9 X = (
10     summary_pl
11     .select(feature_cols)
12     .to_numpy()
13 )
```

A practical workflow may use Polars for fast typed data preparation, pandas for ecosystem compatibility, and NumPy/SciPy/PyTorch for numerical modelling. The boundary between tools should be explicit.

When to choose pandas, when to choose Polars

A useful practical comparison is:

Situation	Usually natural choice
Small exploratory notebook	pandas
Many existing examples or libraries expect pandas	pandas
Large columnar files and repeated filters	Polars
Need lazy query optimization	Polars
Mostly row-wise custom Python logic	neither is ideal; rethink or isolate the kernel
Need numerical arrays for solvers or ML	convert deliberately to NumPy / framework tensors

This is not a competition between libraries. It is a question of matching the tool to the structure of the analysis.

The best workflow is the one whose assumptions, data flow, and performance limits remain visible.

A compact Polars pipeline

The pandas pipeline from this lecture has a direct Polars analogue.

```
1 summary = (  
2     pl.scan_parquet("data/raw/events.parquet")  
3     .join(  
4         pl.scan_parquet("data/raw/runs.parquet"),  
5         on="run_id",  
6         how="left",  
7     ).filter(  
8         (pl.col("energy_GeV") >= 5.0)  
9         & (pl.col("chi2") <= 2.5)  
10        & pl.col("signal").is_not_null()  
11    ).with_columns(  
12        signal_rate=pl.col("signal") / pl.col("exposure_s")  
13    ).group_by(["run_id", "detector"])  
14    .agg(  
15        n_events=pl.len(),  
16        mean_rate=pl.col("signal_rate").mean(),  
17    ).collect())
```

The scientific stages are the same: load, join, filter, derive, aggregate, collect. The execution model is different.

What Polars should leave behind

The most important ideas from Polars are useful even if we later return to pandas:

- ▶ think in typed columns,
- ▶ express transformations as column expressions,
- ▶ delay execution when the whole query can be optimized,
- ▶ read only the rows and columns needed for the result,
- ▶ treat file format, schema, and execution mode as part of the workflow.

For scientific data analysis, Polars is most valuable when the analysis is naturally a pipeline over columns:

scan $\longrightarrow$ filter $\longrightarrow$ derive $\longrightarrow$ join $\longrightarrow$ aggregate $\longrightarrow$ collect.

The result should still be checked, saved, plotted, and interpreted with the same scientific discipline as any other numerical output.

Section 6

Further reading

Supplementary literature

- ▶ McKinney, W. Data Structures for Statistical Computing in Python. In: *Proceedings of the 9th Python in Science Conference*, 56–61 (2010). DOI: <https://doi.org/10.25080/Majora-92bf1922-00a>
- ▶ Wickham, H. Tidy Data. *Journal of Statistical Software* **59**(10), 1–23 (2014). DOI: <https://doi.org/10.18637/jss.v059.i10>
- ▶ Broman, K.W., Woo, K.H. Data Organization in Spreadsheets. *The American Statistician* **72**(1), 2–10 (2018). DOI: <https://doi.org/10.1080/00031305.2017.1375989>
- ▶ Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., Teal, T.K. Good enough practices in scientific computing. *PLOS Computational Biology* **13**(6), e1005510 (2017). DOI: <https://doi.org/10.1371/journal.pcbi.1005510>